PRACTICAL HANDBOOK

Power BI Real-World Projects

A step-by-step guide from raw data to published dashboard, with five complete portfolio-ready projects.

POWER QUERY • DATA MODELING • DAX • DASHBOARDS

For analysts, students and career switchers | 2026 Edition

Contents

01 What You Will Build 3

02 Chapter 1: The Professional Power BI Workflow 4

03 Chapter 2: Setup and Practice Datasets 5

04 Project 1: Sales Performance Dashboard 6

05 Project 2: HR Analytics Dashboard 11

06 Project 3: Finance P&L and Budget Variance Report 14

07 Project 4: Customer Churn and Marketing Analysis 18

08 Project 5: Inventory and Supply Chain Monitor 21

09 Chapter 8: Publish, Refresh, Secure and Share 25

10 Chapter 9: Performance Optimization 27

11 Chapter 10: Design Principles for Professional Dashboards 28

12 Chapter 11: DAX Quick Reference 29

13 Chapter 12: Troubleshooting Guide 30

14 Chapter 13: Build Your Portfolio and Career 31

15 Appendix A: Project Delivery Checklist 32

16 Appendix B: Practice Challenges 33

What You Will Build

This ebook walks you through **five complete, portfolio-ready Power BI projects**, one click at a time. Every project follows the same professional workflow used by BI teams in industry, so by the end you will not just know features, you will know how to deliver.

#	Project	Business Question	Key Skills
1	Sales Performance Dashboard	Which products, regions and months drive revenue?	Star schema, Date table, core DAX, KPI cards
2	HR Analytics Dashboard	Why are employees leaving?	Attrition metrics, age bands, drill-through
3	Finance P&L Report	Are we hitting budget?	Variance DAX, matrix, waterfall chart
4	Customer Churn and Marketing Analysis	Which customers are at risk?	Segmentation, what-if parameter
5	Inventory and Supply Chain Monitor	What will we run out of?	Stock coverage, conditional formatting, alerts

Software needed: Power BI Desktop (free, Windows). A free Power BI Service account is needed for publishing (a work or school email is required for sign-up).

Chapter 1: The Professional Power BI Workflow

Every real project, however small, passes through seven stages. Memorize them; the rest of this book repeats them.

1. **Define** the business questions and the audience.
2. **Acquire** data (Get Data).
3. **Clean and shape** it in Power Query.
4. **Model** it as a star schema with relationships.
5. **Calculate** with DAX measures.
6. **Visualize** with a clear layout and design.
7. **Publish and govern**: share, refresh, secure.

Golden Rules

- **Questions first, charts second.** Write 3 to 5 questions before touching data.
- **Clean in Power Query, calculate in DAX.** Do transformations upstream, not in visuals.
- **Use measures, not calculated columns,** whenever possible. Measures are faster and lighter.
- **One fact table, several dimension tables.** This is the star schema.
- **Less is more.** Aim for 6 to 8 visuals per page.

The Star Schema in Words

A central **fact table** holds events with numbers (sales, hires, transactions). Around it sit **dimension tables** holding descriptions (Date, Product, Customer, Region). Dimensions filter the fact table through **one-to-many relationships**, with the filter arrow pointing from the dimension to the fact.

Chapter 2: Setup and Practice Datasets

Step 1: Install Power BI Desktop

1. Open the Microsoft Store on Windows and search for **Power BI Desktop**, or download it from the official Microsoft page.
2. Install and launch it. Sign in if you wish.
3. Go to **File > Options and settings > Options > Data Load** and turn **Time intelligence (auto date/time)** off. You will build your own Date table.

Step 2: Get Practice Data

Use free public datasets so you can publish your work legally:

- **Microsoft Contoso / AdventureWorks sample data** (sales).
- **Kaggle:** search for *IBM HR Analytics Employee Attrition*, *Telco Customer Churn*, *Superstore Sales*.
- **Your own CSV or Excel exports** from any business you know.

Create a project folder with this structure:

```
PowerBI-Projects/
  01-Sales/      data, pbix, screenshots
  02-HR/
  03-Finance/
  04-Churn/
  05-Inventory/
```

Step 3: Learn the Four Views

- **Report view:** build visuals.
- **Table view:** inspect data.
- **Model view:** manage relationships.
- **DAX query view:** test measures.

Project 1: Sales Performance Dashboard

Scenario: You are the analyst at a retail company. The Sales Director wants to know revenue, profit, best products, strongest regions, and monthly trends compared with last year.

Business questions:

1. What are total sales, profit, and margin?
2. How are sales trending month by month and year over year?
3. Which products and categories perform best?
4. Which regions and customers contribute most?

Data (example Superstore-style): one Orders table with *Row ID, Order ID, Order Date, Ship Date, Customer ID, Customer Name, Segment, Country, Region, State, City, Product ID, Category, Sub-Category, Product Name, Sales, Quantity, Discount, Profit*. If your file has no Customer ID or Product ID, build a dimension from the cleaned unique names and add an Index column to that small table, then merge the index back into Orders so both tables share it.

Phase A: Load the Data

1. **Home > Get data > Excel workbook** (or Text/CSV).
2. Select the file, then pick the Orders sheet and click **Transform Data** (not Load).

Phase B: Clean in Power Query

1. Check column data types. Click the icon at each header: set dates to **Date**, Sales and Profit to **Fixed decimal number**, Quantity to **Whole number**, Discount to **Decimal number**.
2. **Handle duplicates carefully.** Two different problems look alike:
   - *Repeated source rows* (the same line exported twice). If the data has an order-line identifier such as `Row ID`, select only that column and use **Remove Rows > Remove Duplicates**. If it has none, do not guess: an order can legitimately contain the same product on two lines, so never de-duplicate on Order ID plus Product ID. Only remove fully identical rows (all columns selected) after confirming with the data owner that they are export errors.
   - *Missing keys.* An **Index Column** is a surrogate key. It gives every row a different number, so it cannot detect or remove duplicates. Add it only after cleaning, and only when you need a unique key.
3. **Handle blanks:** filter each key column and remove rows where Sales is null. Replace null Discount with 0 using **Transform > Replace Values**.
4. **Trim text:** select text columns, **Transform > Format > Trim** and **Clean**. Use **Capitalize Each Word** for names.
5. **Rename the query** to `Orders` (right pane, Properties).

6. **Create dimension tables** by referencing, not duplicating:
   - Right-click the Orders query > **Reference**. Rename to `Products`.
   - Keep only Product ID (or Product Name), Sub-Category, Category. **Remove Duplicates**.
   - Repeat for `Customers` (Customer ID, Name, Segment).
   - For `Geography`, build a key that matches the dataset's grain. Decide the lowest level you need (city, or postal code if the data has it), then merge every column needed to make it unique, for example Country, State and City, with **Transform > Merge Columns**, separator `-`, named `Geo Key`. A data set covering several countries needs Country in the key, because the same state or city name can repeat. Reference Orders again, keep Geo Key plus Country, Region, State, City, and **Remove Duplicates** on Geo Key. **Test it:** in Power Query turn on **View > Column profile** and confirm that Geo Key has as many distinct values as rows. Region alone is never a safe key.
   - Reference Orders once more, keep only `Region` (add `Country` too if the data spans countries), **Remove Duplicates**, and name it `Regions`. This small unique list is used for row-level security in Chapter 8.
7. In `Orders`, remove the descriptive columns that moved to dimensions, keeping only the keys, dates, and numbers.
8. Click **Close & Apply**.

> **Check:** In Power Query the Applied Steps pane should read like a recipe: Source, Changed Type, Removed Duplicates, Trimmed Text. Clear step names help future you.

Phase C: Build the Date Table

A proper Date table is mandatory for time intelligence.

1. **Table view > New table**, then enter the formula shown after these steps.
2. Select the `Month` column, **Column tools > Sort by column > Month Number**. This keeps months in calendar order.
3. Right-click the Date table > **Mark as date table**, choose the Date column.

Date table formula (step 1):

```
Date =
ADDCOLUMNS (
    CALENDAR ( DATE ( 2020, 1, 1 ), DATE ( 2025, 12, 31 ) ),
    "Year", YEAR ( [Date] ),
    "Month Number", MONTH ( [Date] ),
    "Month", FORMAT ( [Date], "MMM" ),
    "Quarter", "Q" & QUARTER ( [Date] ),
    "Year-Month", FORMAT ( [Date], "YYYY-MM" )
)
```

> **Tip:** Set the date range to cover your entire dataset; use MIN and MAX of Order Date if you want it dynamic.

Phase D: Model the Data

1. Open **Model view**.
2. Drag `Date[Date]` onto `Orders[Order Date]`. Confirm: **One to many (1:*)**, single direction.
3. Drag `Products[Product ID]` to `Orders[Product ID]`, `Customers[Customer ID]` to `Orders[Customer ID]`, `Geography[Geo Key]` to `Orders[Geo Key]`, and `Regions[Region]` to `Geography[Region]` (one to many, single direction).
4. Make sure every dimension key is **unique**. If Power BI warns about many-to-many, return to Power Query and remove duplicates in the dimension.
5. Hide foreign-key columns in Orders from Report view (right-click > Hide) so report authors use dimension fields.

Phase E: Write the DAX Measures

Create a blank table named `_Measures` (Home > Enter data, rename) to keep measures organized. Then use **New measure** for each:

```
Total Sales = SUM ( Orders[Sales] )

Total Profit = SUM ( Orders[Profit] )

Profit Margin % = DIVIDE ( [Total Profit], [Total Sales] )

Total Orders = DISTINCTCOUNT ( Orders[Order ID] )

Avg Order Value = DIVIDE ( [Total Sales], [Total Orders] )

Sales LY =
CALCULATE ( [Total Sales], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

Sales YoY % =
DIVIDE ( [Total Sales] - [Sales LY], [Sales LY] )

Sales YTD =
TOTALYTD ( [Total Sales], 'Date'[Date] )
```

Format each: select the measure, **Measure tools** > set format (Currency, Percentage with 1 decimal).

> **Why DIVIDE?** It safely returns blank instead of an error when the denominator is zero.

Phase F: Design the Report Page

Layout (16:9, one page):

- **Top strip:** title on the left, slicers (Year, Region, Category) on the right.
- **Row 1:** four KPI **Card** visuals: Total Sales, Total Profit, Profit Margin %, Total Orders.
- **Row 2 left: Line chart**, X-axis `Date[Year-Month]`, Y-axis Total Sales and Sales LY.
- **Row 2 right: Clustered bar**, Axis Category, Values Total Sales.
- **Row 3 left: Filled map** or bar by Region, with profit as color saturation.
- **Row 3 right: Table** of top 10 products (apply a Top N filter by Total Sales).

Step by step:

1. **Canvas settings:** View > Page view > Fit to page. Format pane > Canvas background: light gray (#F4F6F8), transparency 0. (For a dark report, pick a deep navy such as #0B1020.)
2. Insert a **rectangle** shape for each section, with a subtle shadow, to create card panels.
3. Insert the four cards. Under **Format > Callout value** set size to 28.
4. Insert the line chart. Add `Date[Year-Month]` to X-axis; add Total Sales and Sales LY to Y-axis. Turn off gridlines; keep the legend at the top.
5. Insert **Slicers**. Use the **Dropdown** style for Region and Category, and **Between** style for Year. Sync slicers across pages via **View > Sync slicers**.
6. **Top N:** select the table, in the **Filters** pane click the product field > Filter type **Top N** > Show items **Top 10** by Total Sales > **Apply filter**.
7. **Conditional formatting:** in the table, click the dropdown on Profit Margin % > **Conditional formatting > Data bars** or **Font color** by rules (below 0% red, above 15% green).
8. **Titles and alt text:** give every visual a clear title (for example, Monthly Sales vs Last Year) and fill in **Alt text** for accessibility.

Phase G: Add Interactivity

- **Edit interactions** (Format > Edit interactions) to stop a visual from filtering another when you want a fixed context.
- **Tooltips:** create a hidden page, set **Page information > Allow use as tooltip**, set page size to Tooltip, then assign it in a visual's **Tooltip** settings.
- **Bookmarks and buttons:** add a **Reset filters** button using a bookmark captured with all slicers cleared.
- **Drill-through:** create a `Product Detail` page, drag `Products[Product Name]` into **Drill through** fields, add details; right-click any product on the main page to jump there.

Phase H: Quality Check Before Publishing

- Do totals reconcile with the source (compare to an Excel pivot)?
- Does the Year slicer change every visual correctly?
- Are all numbers formatted (thousands separators, currency)?

- Is text readable at 100% zoom, with a minimum size of 10 pt?
- Is the page free of clutter, with consistent colors?

Deliverables: `Sales_Dashboard.pbix`, two screenshots, and a one-paragraph business summary of your findings.

Project 2: HR Analytics Dashboard

Scenario: The HR Director is worried about attrition (employees leaving). She needs to see who is leaving, where, and likely why.

Business questions:

1. What is the overall attrition rate?
2. Which departments, roles and age groups have the highest attrition?
3. Does salary, overtime, or tenure influence leaving?
4. What does the current workforce look like by department, tenure and age?

Data (IBM HR Analytics dataset): *EmployeeNumber, Age, Attrition (Yes/No), Department, JobRole, MonthlyIncome, OverTime, YearsAtCompany, Gender, JobSatisfaction, DistanceFromHome*.

Phase A: Prepare Data

1. **Get data > Text/CSV**, then **Transform Data**.
2. Set data types. Ensure Attrition and OverTime are Text.
3. **Add a numeric flag: Add Column > Conditional Column**. Name it `Left Flag`; if Attrition equals Yes then 1, otherwise 0.
4. **Create age bands:** a Conditional Column `Age Group`: below 25 = `Under 25`, below 35 = `25-34`, below 45 = `35-44`, below 55 = `45-54`, else `55+`.
5. **Create tenure bands** the same way on YearsAtCompany: `0-2`, `3-5`, `6-10`, `10+`.
6. **Create income bands** from MonthlyIncome (Low, Medium, High) using quartiles (**Transform > Statistics**).
7. **Close & Apply.**

> **Sorting tip:** Text bands sort alphabetically. Add a `Sort Order` column (1 to 5) and use **Sort by column** to display them in the right order.

Phase B: Measures

```
Headcount = COUNTROWS ( Employees )

Leavers = SUM ( Employees[Left Flag] )

Attrition Rate = DIVIDE ( [Leavers], [Headcount] )

Active Employees = [Headcount] - [Leavers]

Avg Monthly Income = AVERAGE ( Employees[MonthlyIncome] )

Avg Tenure = AVERAGE ( Employees[YearsAtCompany] )

Overtime Attrition Rate =
CALCULATE (
    [Attrition Rate],
    Employees[OverTime] = "Yes"
)
```

Phase C: Build the Visuals

Page 1, Executive Overview

1. **Cards:** Headcount, Leavers, Attrition Rate (format 0.0%), Avg Tenure.
2. **Donut chart:** Legend Attrition, Values Headcount.
3. **Bar chart:** Axis Department, Values Attrition Rate. Sort descending.
4. **Column chart:** Axis Age Group (sorted), Values Attrition Rate.
5. **Matrix:** Rows JobRole, Values Headcount, Leavers, Attrition Rate. Apply a background gradient on Attrition Rate.

Page 2, Drivers of Attrition

1. **Clustered column:** OverTime vs Attrition Rate.
2. **Scatter chart:** X Avg Monthly Income, Y Attrition Rate, Details JobRole, Size Headcount.
3. **Line chart:** Axis Tenure Band, Values Attrition Rate.
4. **Key influencers visual:** Analyze Attrition (select Yes), Explain by OverTime, JobSatisfaction, DistanceFromHome, Age, MonthlyIncome.
5. **Decomposition tree:** Analyze Leavers, Explain by Department, JobRole, Gender for a drill-down story.

Page 3, Employee Detail (drill-through): a table with Employee Number, Role, Income, Tenure, Satisfaction. Add Department to drill-through.

Phase D: Insights to Write Down

Examples of statements a good analyst states (verify on your own data):

- Attrition is concentrated in specific roles and among employees working overtime.
- Younger employees and early-tenure staff leave at higher rates.
- Lower income correlates with higher leaving rates in several roles.

End with **recommendations**: review overtime policy, create a retention program for the first two years, and benchmark pay for high-attrition roles.

> **Data limitation:** the IBM attrition file is a single snapshot with no hire or exit dates, so it cannot show a real headcount trend over time. Describe the current workforce and attrition by segment, and do not build a month-by-month line chart from it. A true trend needs hire and termination dates, or monthly headcount snapshots from your HR system.
> **Ethics note:** HR data is sensitive. Anonymize names, use row-level security (see Chapter 8), and never publish personal data publicly.

Project 3: Finance P&L and Budget Variance Report

Scenario: The CFO needs a monthly Profit and Loss (P&L) report comparing **Actual** against **Budget**, with clear variance explanations.

Business questions:

1. What are revenue, cost of goods sold (COGS), gross profit, operating expenses and net profit?
2. How do actuals compare to budget, in amount and percentage?
3. Which accounts and departments cause the biggest variances?

Data: four tables.

- `Actuals` : *Date, Account ID, Department ID, Amount*
- `Budget` : *Date, Account ID, Department ID, Amount*
- `Accounts` : *Account ID, Account Name, Category (Revenue, COGS, OpEx), Sort Order*
- `Departments` : *Department ID, Department Name*

Phase A: Load and Prepare

1. Load all four tables via **Get data** and open **Transform Data**.
2. In Actuals and Budget, set Date to **Date**, Amount to **Fixed decimal**.
3. Add a column `Scenario` (**Add Column > Custom Column**): value `Actual` in one table, `Budget` in the other.
4. **Append** the two fact tables: **Home > Append Queries as New** and name it `Financials`.
5. In Accounts, ensure Sort Order is numeric. Keep revenue and costs both positive, and let DAX handle the logic.
6. Disable load for the original Actuals and Budget queries (right-click > uncheck **Enable load**).
7. **Close & Apply**, create the Date table as in Project 1, and relate Date, Accounts and Departments to Financials (one-to-many).

Phase B: Measures

```
Actual = CALCULATE ( SUM ( Financials[Amount] ), Financials[Scenario] = "Actual" )

Budget = CALCULATE ( SUM ( Financials[Amount] ), Financials[Scenario] = "Budget" )

Variance = [Actual] - [Budget]

Variance % = DIVIDE ( [Variance], [Budget] )

Revenue = CALCULATE ( [Actual], Accounts[Category] = "Revenue" )

COGS = CALCULATE ( [Actual], Accounts[Category] = "COGS" )

Gross Profit = [Revenue] - [COGS]

Gross Margin % = DIVIDE ( [Gross Profit], [Revenue] )

OpEx = CALCULATE ( [Actual], Accounts[Category] = "OpEx" )

Net Profit = [Gross Profit] - [OpEx]

Net Profit YTD = TOTALYTD ( [Net Profit], 'Date'[Date] )

Favorable Variance =
SUMX (
    VALUES ( Accounts[Category] ),
    VAR v = [Actual] - [Budget]
    RETURN
        IF ( Accounts[Category] IN { "COGS", "OpEx" }, -v, v )
)

Variance Status =
SWITCH (
    TRUE (),
    [Variance %] >= 0.05, "Above plan",
    [Variance %] <= -0.05, "Below plan",
    "On plan"
)
```

> **Note:** For cost accounts, spending above budget is unfavorable. Favorable Variance evaluates each category separately (SUMX over VALUES), flips the sign for COGS and OpEx, then adds the results. That makes it correct at every level: a single account, a category, or the grand total, where it equals the Net Profit variance. Use it, not raw Variance, for red and green formatting.

Phase C: Build the Report

1. **Cards:** Revenue, Gross Profit, Net Profit, Gross Margin %.
2. **Matrix (the P&L statement):** Rows Accounts[Category] then Accounts[Account Name]; Columns Date[Year] or Date[Month]; Values Actual, Budget, Variance, Variance %. Turn on **Expand all down one level** and **Row subtotals**.
3. **Conditional formatting:** on a Favorable Variance column, in the matrix and any table, use **Font color > Rules**: below 0 red, 0 and above green. Never color raw Variance for cost lines.
4. **Waterfall chart (Revenue to Net Profit bridge):** raw Actual values are all positive, so a waterfall would add costs instead of subtracting them. Build a signed bridge instead. Create a small table with **Enter data**, named `P&L Bridge`, with one column `Step` containing Revenue, COGS, OpEx, plus a `Sort` column (1, 2, 3), and sort Step by Sort. Do not relate the table to anything. Add the `Bridge Value` measure shown after this list, then set the waterfall's Category to `P&L Bridge[Step]` and Y-axis to `Bridge Value`. The visual adds a final Total bar automatically, which equals Net Profit. Native waterfalls cannot show a mid-chart subtotal, so show Gross Profit in an adjacent card.
5. **Clustered column:** Axis Date[Month], Values Actual and Budget.
6. **Bar chart:** Axis Department Name, Values Variance for the top overspending departments.
7. **Slicers:** Year, Month, Department.
8. **Smart narrative:** Insert > **Smart narrative** to auto-generate a text summary, then edit it.

Bridge measure for step 4:

```
Bridge Value =
SWITCH (
    SELECTEDVALUE ( 'P&L Bridge'[Step] ),
    "Revenue", [Revenue],
    "COGS", -[COGS],
    "OpEx", -[OpEx]
)
```

Phase D: Finishing Touches

- Use a finance color scheme: navy or gold for actual, gray for budget, red and green only for variance.
- Format values in thousands (K) or millions (M) with **Display units**.

- Add a **tooltip page** with a variance breakdown.

> **Check:** Revenue minus COGS minus OpEx must equal Net Profit. Reconcile against the general ledger totals.

Project 4: Customer Churn and Marketing Analysis

Scenario: A telecom or subscription company is losing customers. Marketing wants to know who is at risk and what a retention campaign could save.

Business questions:

1. What is the churn rate, and how has it moved?
2. Which segments (contract, tenure, payment method, plan) churn most?
3. How much revenue is at risk?
4. What savings would a 5% reduction in churn deliver?

Data (Telco Customer Churn): *customerID, gender, SeniorCitizen, tenure, Contract, PaymentMethod, InternetService, MonthlyCharges, TotalCharges, Churn*.

Phase A: Clean

1. **Get data > Text/CSV > Transform Data.**
2. TotalCharges often loads as text with blanks. Change the type to **Decimal number**, then **Replace Errors** with 0 (right-click the column header).
3. Convert SeniorCitizen from 0/1 to `No` / `Yes` using **Replace Values**.
4. Add `Churn Flag` (1 if Churn is Yes, else 0).
5. Add `Tenure Group`: `0-12`, `13-24`, `25-48`, `49+` months, with a `Tenure Sort` column.
6. Add `Monthly Charge Band`: Low (below 35), Medium (35 to 70), High (above 70).
7. **Close & Apply**.

Phase B: Measures

```
Customers = COUNTROWS ( Customers )

Churned Customers = SUM ( Customers[Churn Flag] )

Churn Rate = DIVIDE ( [Churned Customers], [Customers] )

Monthly Recurring Revenue = SUM ( Customers[MonthlyCharges] )

Revenue at Risk =
CALCULATE ( [Monthly Recurring Revenue], Customers[Churn] = "Yes" )

Avg Billed per Customer = AVERAGE ( Customers[TotalCharges] )

Avg Monthly Charge = AVERAGE ( Customers[MonthlyCharges] )

Monthly Churn Rate =
DIVIDE ( [Churned Customers], SUM ( Customers[tenure] ) )

Estimated CLV = DIVIDE ( [Avg Monthly Charge], [Monthly Churn Rate] )
```

Billed amount is not lifetime value. Avg Billed per Customer is only the average historical total billed to date, and it includes customers still active. Estimated CLV is a simple model: average monthly charge divided by monthly churn, where monthly churn is churned customers divided by total customer-months (the sum of tenure). It ignores cost to serve, margin and discounting, so label it an estimate and use it for comparing segments, not as an accounting figure.

Phase C: What-If Parameter

1. **Modeling > New parameter > Numeric range.**
2. Name it `Churn Reduction`, minimum 0, maximum 0.20, increment 0.01, default 0.05. Tick **Add slicer**.
3. Power BI creates a table and a measure `Churn Reduction Value`.
4. Create the scenario measures:

```
Revenue Saved = [Revenue at Risk] * [Churn Reduction Value]

Projected Churn Rate = [Churn Rate] * ( 1 - [Churn Reduction Value] )
```

1. Add cards for both, and let stakeholders move the slider live in a meeting.

Phase D: Visuals

1. **Cards:** Customers, Churn Rate, Revenue at Risk, Revenue Saved.
2. **100% stacked bar:** Axis Contract, Legend Churn.
3. **Column chart:** Axis Tenure Group, Values Churn Rate.
4. **Treemap:** Group PaymentMethod, Values Churned Customers.
5. **Matrix heatmap:** Rows InternetService, Columns Contract, Values Churn Rate, with a background color gradient.
6. **Key influencers:** analyze Churn equals Yes.
7. **Slicers:** Gender, Senior Citizen, Monthly Charge Band.

Phase E: Findings Template

- **What we see:** the segments with the highest churn rate.
- **Why it matters:** revenue at risk in currency terms.
- **What we recommend:** for example, offer discounts for annual contracts, improve first-year onboarding, and encourage automatic payment.

Project 5: Inventory and Supply Chain Monitor

Scenario: The operations manager wants to avoid stockouts and overstock, and to see supplier delivery performance.

Business questions:

1. What is current stock value and stock coverage (days)?
2. Which items are below reorder level?
3. Which suppliers deliver late?
4. How fast does stock turn?

Data: `Inventory` (Date, SKU, Stock On Hand, Unit Cost), `Sales` (Date, SKU, Units Sold), `Products` (SKU, Name, Category, Reorder Level, Supplier ID), `Suppliers`, `Purchase Orders` (PO ID, Supplier ID, Order Date, Promised Date, Received Date, Quantity).

Phase A: Prepare

1. Load all tables; set types.
2. In Purchase Orders, add `Days Late` = Received Date minus Promised Date (**Add Column > Custom Column**, then change type to Whole number). Add `On Time Flag` (1 if Days Late is 0 or less).
3. Relate Products to Inventory, Sales and Purchase Orders (via SKU); relate Suppliers to Products; connect Date to each fact table.

Phase B: Measures

```
Units Sold = SUM ( Sales[Units Sold] )

Stock On Hand =
VAR PeriodEnd = MAX ( 'Date'[Date] )
RETURN
    SUMX (
        VALUES ( Products[SKU] ),
        VAR LastSnap =
            CALCULATE (
                MAX ( Inventory[Date] ),
                REMOVEFILTERS ( 'Date' ),
                Inventory[Date] <= PeriodEnd
            )
        RETURN
            CALCULATE (
                SUM ( Inventory[Stock On Hand] ),
                REMOVEFILTERS ( 'Date' ),
                Inventory[Date] = LastSnap
            )
    )

Stock Value =
VAR PeriodEnd = MAX ( 'Date'[Date] )
RETURN
    SUMX (
        VALUES ( Products[SKU] ),
        VAR LastSnap =
            CALCULATE (
                MAX ( Inventory[Date] ),
                REMOVEFILTERS ( 'Date' ),
                Inventory[Date] <= PeriodEnd
            )
        RETURN
            CALCULATE (
                SUMX ( Inventory, Inventory[Stock On Hand] * Inventory[Unit Cost] ),
                REMOVEFILTERS ( 'Date' ),
                Inventory[Date] = LastSnap
            )
    )

Avg Daily Sales =
DIVIDE ( [Units Sold], DISTINCTCOUNT ( 'Date'[Date] ) )

Days of Coverage = DIVIDE ( [Stock On Hand], [Avg Daily Sales] )
```

```
Below Reorder =
IF ( [Stock On Hand] < MAX ( Products[Reorder Level] ), 1, 0 )

On-Time Delivery % =
DIVIDE (
    SUM ( 'Purchase Orders'[On Time Flag] ),
    COUNTROWS ( 'Purchase Orders' )
)
```

How the snapshot measures work. Inventory is a balance at a point in time, so summing across days is wrong. For each SKU the measure finds its latest snapshot on or before the end of the selected period (an as-of value), takes that day's quantity, and adds the SKUs together. A SKU with no count in the period therefore uses its most recent earlier count, and a SKU with no snapshot at all returns blank. Use Below Reorder at SKU level, where Reorder Level is a single value.

Test these measures before trusting them. Compare each result with a manual check in Excel:

- Select one month: each SKU should match its last snapshot on or before that month-end.
- Select a period where one SKU has no count: it should show its earlier snapshot, not zero.
- Add a SKU with no snapshots: it should show blank and not break totals.
- Clear all date filters: totals should equal each SKU's latest snapshot overall.
- Use two SKUs whose latest counts fall on different dates: the total must equal the sum of both.

Phase C: Visuals

1. **Cards:** Stock Value, Days of Coverage, On-Time Delivery %.
2. **Alert table:** SKU, Product, Stock On Hand, Reorder Level, Days of Coverage. Apply **Background color > Rules**: red when Days of Coverage is below 7, amber below 14, green otherwise.
3. **Line chart:** Stock On Hand over time with Units Sold on a secondary axis.
4. **Bar chart:** Supplier vs On-Time Delivery %.
5. **Scatter:** X Units Sold, Y Days of Coverage, Details Category, to spot slow movers.
6. **Gauge:** On-Time Delivery % with a 95% target.
7. **Filter pane:** visual filter on the table where Below Reorder equals 1.

Phase D: Data Alerts

After publishing, pin a **Card** to a dashboard in the Service, click **...** > **Manage alerts**, and set a threshold such as Days of Coverage below 7 to receive email or mobile notifications.

Chapter 8: Publish, Refresh, Secure and Share

Step 1: Publish to the Power BI Service

1. In Desktop, click **Home > Publish**.
2. Sign in and choose a **Workspace** (My workspace for practice; a team workspace in real work).
3. Click the **Open in Power BI** link to view it online.

Step 2: Schedule Data Refresh

1. In the Service, hover over the semantic model (dataset) > **Settings**.
2. For on-premises sources or files on your PC, install a **gateway** from **Settings > Manage gateways**.
3. Under **Data source credentials**, sign in.
4. Under **Scheduled refresh**, turn it on, choose frequency and time zone, and add a failure-notification email.
5. Cloud sources such as SharePoint Online or Azure refresh without a gateway.

Step 3: Row-Level Security (RLS)

RLS restricts rows by user, for example each regional manager sees only their region.

1. In Desktop: **Modeling > Manage roles > Create**.
2. Create a `Security` table (**Enter data**) with columns `Email` and `Region`. One row per user and region, so a region can have many users and a user can have several regions. Hide the table from Report view. Do not create relationships for it.
3. Use the `Regions` table from Project 1 (unique list of regions, related one to many to Geography, which is related to Orders). Create the role `Region Manager`, select the `Regions` table and enter the filter shown after this list. Filters flow from Regions to Geography to Orders along the existing one-to-many relationships, so no bi-directional relationship is needed. Add a new user by adding a row to Security.
4. **Test:** Modeling > **View as** > tick `Region Manager` and **Other user**, then enter an email from the Security table. Test a user with one region, a user with two regions, and an email that is not listed (that user should see no data).
5. After publishing, open the semantic model > **Security** and add users or groups to the role. RLS applies to workspace Viewers only; Admins, Members and Contributors see all data, so share through an App or give people Viewer access.

Role filter for step 3:

```
Regions[Region]
    IN SELECTCOLUMNS (
        FILTER ( Security, Security[Email] = USERPRINCIPALNAME () ),
        "Region", Security[Region]
    )
```

Step 4: Share Safely

- **Share** a report link with named people (viewers need Pro or Premium Per User licensing unless the workspace is on Fabric/Premium capacity).
- Create an **App** to distribute a polished, versioned bundle of reports to a wider audience.
- Use **Publish to web** only for truly public data; it is open to anyone with the link.
- Use **Export > PDF or PowerPoint** for static copies.

Step 5: Mobile View

In Desktop, **View > Mobile layout**, drag key visuals onto the phone canvas, and test in the Power BI mobile app.

Chapter 9: Performance Optimization

Problem	Fix
Slow report	Remove unused columns and tables; reduce visuals per page
Large file	Turn off auto date/time; use integers and avoid high-cardinality text columns
Slow DAX	Use measures instead of calculated columns; avoid iterators over large tables where an aggregate works
Slow refresh	Filter data at the source; use query folding (right-click a step > View Native Query) and incremental refresh

Use Performance Analyzer: View > Performance analyzer > Start recording > Refresh visuals. It shows time per visual, split into DAX query, visual display, and other.

Use incremental refresh for big fact tables: create `RangeStart` and `RangeEnd` parameters (DateTime), filter the date column using them, then define the policy under the table's **Incremental refresh** menu.

Chapter 10: Design Principles for Professional Dashboards

1. **Visual hierarchy:** the most important KPI top-left. Readers scan in an F pattern.
2. **Consistent palette:** one primary color, one accent, plus red and green for status. Never more than 5 colors.
3. **Choose the right chart:** trend means line; compare categories means bar; part-to-whole with few parts means donut; relationship means scatter; ranking means sorted bar. Avoid 3D and pies with many slices.
4. **Whitespace and alignment:** use **Format > Align** and **Distribute**; keep margins equal.
5. **Labels that tell the story:** action titles such as *Sales grew 12% year over year* beat generic ones like *Sales chart*.
6. **Accessibility:** contrast ratio above 4.5:1, alt text, tab order, no color-only meaning.
7. **Use a theme:** build or import a JSON theme (View > Themes > Browse for themes) so every report looks the same. A dark navy and gold theme gives a premium feel.

Chapter 11: DAX Quick Reference

Goal	Function	Example
Sum / average	`SUM` , `AVERAGE`	`SUM ( Orders[Sales] )`
Count rows / unique	`COUNTROWS` , `DISTINCTCOUNT`	`DISTINCTCOUNT ( Orders[Order ID] )`
Change filter context	`CALCULATE`	`CALCULATE ( [Sales], Orders[Region] = "East" )`
Remove filters	`ALL` , `ALLSELECTED`	`CALCULATE ( [Sales], ALL ( Products ) )`
Row-by-row math	`SUMX` , `AVERAGEX`	`SUMX ( Orders, Orders[Qty] * Orders[Price] )`
Prior year	`SAMEPERIODLASTYEAR`	`CALCULATE ( [Sales], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )`
Year to date	`TOTALYTD`	`TOTALYTD ( [Sales], 'Date'[Date] )`
Rolling 3 months	`DATESINPERIOD`	`CALCULATE ( [Sales], DATESINPERIOD ( 'Date'[Date], MAX ( 'Date'[Date] ), -3, MONTH ) )`
Ranking	`RANKX`	`RANKX ( ALL ( Products[Name] ), [Sales] )`
Conditions	`IF` , `SWITCH`	`SWITCH ( TRUE (), [Margin] > 0.2, "High", "Low" )`
Safe division	`DIVIDE`	`DIVIDE ( [Profit], [Sales], 0 )`

```
Sales Growth % =
VAR CurrentSales = [Total Sales]
VAR PriorSales = [Sales LY]
RETURN
    DIVIDE ( CurrentSales - PriorSales, PriorSales )
```

Understand context: *Row context* is the current row (calculated columns, iterators). *Filter context* is the set of filters from slicers, visuals and relationships. CALCULATE modifies filter context, which is the most important DAX idea to master.

Chapter 12: Troubleshooting Guide

Symptom	Likely Cause	Fix
Totals look too large	Many-to-many or duplicate keys in a dimension	Remove duplicates; check relationship cardinality
Slicer does nothing	No relationship or inactive relationship	Check Model view; use USERELATIONSHIP for inactive ones
Time intelligence returns blank	Date table not marked or gaps in dates	Mark as date table; use a continuous calendar
Month names out of order	Missing sort-by column	Sort Month by Month Number
Same value in every row	Field comes from the wrong table	Use a dimension field on the axis
Refresh fails in the Service	Expired credentials or gateway offline	Re-enter credentials; check gateway status
Percentage shows 0.2 instead of 20%	Format not set	Measure tools > Percentage

Chapter 13: Build Your Portfolio and Career

Package Each Project

1. **Title and business problem** (2 sentences).
2. **Data source** and any cleaning steps.
3. **Data model screenshot.**
4. **Dashboard screenshots** (2 to 3 images).
5. **Key insights and recommendations** (3 to 5 bullets).
6. **DAX highlights:** show 2 or 3 interesting measures.

Where to Publish

- **GitHub:** a repository per project with README, PBIX, and screenshots.
- **LinkedIn:** a post with a screenshot and 3 insights.
- **Microsoft Fabric Community / Data Stories Gallery:** share and get feedback.
- **Personal site or Notion page** as a portfolio index.

Interview Talking Points

- Explain your star schema and why you chose it.
- Describe one performance problem you solved.
- Walk through a DAX measure and the filter context behind it.
- Describe how you validated numbers with the business.
- Mention security and refresh decisions.

Certification Path

Microsoft PL-300: Power BI Data Analyst covers Power Query, modeling, DAX, visualization, and deployment, which maps closely to the projects in this book. Check Microsoft Learn for the current exam outline.

Appendix A: Project Delivery Checklist

- Business questions written and agreed
- Data sources documented
- Power Query steps named and clean
- Star schema with a marked Date table
- Measures stored in a measures table, formatted
- Totals reconciled with the source
- Visuals titled, aligned, accessible
- RLS tested with View as
- Published, refresh scheduled, owners assigned
- Documentation and screenshots saved

Appendix B: Practice Challenges

1. Add a **forecast** (Analytics pane) to the Sales line chart and explain its confidence interval.
2. Create a **Pareto (80/20) analysis** of products by sales with a cumulative percentage measure.
3. Build an **RFM customer segmentation** (Recency, Frequency, Monetary) with calculated tiers.
4. Add a **dynamic title** measure such as *Sales for Region: East* using SELECTEDVALUE.
5. Create a **field parameter** so users can switch the chart between Sales, Profit, and Orders.

Congratulations. You now have five finished projects, a repeatable workflow, and a portfolio to show for it. Keep building, keep validating your numbers, and always start with the business question.